Week 15 (Days 99-105): Pretrained models, fine-tuning and LoRA

Status: written in full (Phase 4). Days 102-104 follow the revised plan (peft/QLoRA, instruction tuning, evaluation of the fine-tuned model). `SYLLABUS.md` now lists these titles; the RLHF overview moved to Day 106 (Week 16). Every number below was printed by a script in `code/week-15/` on this box (torch 2.15.0+cpu, transformers 5.21.1, peft 0.21.2, 8 CPU threads, no GPU), with logs in `code/week-15/out/*.log`. Times were measured with the CPU otherwise idle; on a shared CPU they were up to ~100x slower, so treat them as rough.

Week goals

- Load, inspect, run and score a real pretrained LM with Hugging Face `transformers`.
- Fully fine-tune a 135M-parameter model on CPU and understand its memory cost.
- Implement LoRA from scratch, verify it against `peft`, and merge adapters.
- Understand QLoRA (4-bit frozen base + LoRA) and simulate it on CPU.
- Instruction-tune a base model with a chat template and prompt-loss masking.
- Evaluate a fine-tuned model honestly: task metric, forgetting, multiple-choice scoring, judge bias.
- Weekly project: compare full fine-tuning with LoRA variants on the same budget.

```
flowchart LR
  A[Day 99 HF basics] --> B[Day 100 full fine-tune]
  B --> C[Day 101 LoRA from scratch + peft]
  C --> D[Day 102 peft configs + QLoRA]
  D --> E[Day 103 instruction tuning]
  E --> F[Day 104 evaluation]
  F --> G[Day 105 project + quiz]
```

Compute note (CPU vs GPU). A training step of SmolLM2-135M on 4 x 128 tokens took about 0.9 s here (full fine-tune) or 0.5 s (LoRA). That is enough for a hundred steps, which shows the *effects*, but real fine-tunes run thousands of steps on GPUs with bf16 and larger batches. On a GPU you could also use real 4-bit QLoRA (bitsandbytes kernels are GPU-only; Day 102 simulates the quantisation instead) and models of 1-8B parameters.

Models and data (all downloads verified on this box, cached in `data/cache/hf`; scripts run with `HF_HUB_OFFLINE=1`): - `HuggingFaceTB/SmolLM2-135M` (base) and `HuggingFaceTB/SmolLM2-135M-Instruct`, Apache-2.0. - Tiny Shakespeare (as Week 14). In SmolLM tokens: 305,111 train, 35,983 val. - Synthetic "reverse the words" instructions generated by `day103_instruction_tuning.py` (no download).

Shared code: `_hf.py` (cache path, loading, Shakespeare token batches, windowed perplexity) and `_lora.py` (LoRA from scratch).

A note on perplexity numbers. `_hf.perplexity` averages over the first n windows of 256 validation tokens. Day 99 uses 16 windows (24.42); Days 100-105 use 8 windows (26.29 for the same base model). Only compare numbers computed with the same window count.

Day 99: Hugging Face transformers and datasets

Review today: Day 98 (+1), Day 96 (+3), Day 92 (+7), Day 78 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The Hub. The Hugging Face Hub hosts model repositories: a `config.json` (architecture), weights (`model.safetensors`), tokenizer files, and a model card (license, training data, intended use, limitations). Always read the card and the license before use.

The three objects. - `AutoTokenizer.from_pretrained(name)`: text ? token ids, plus special tokens and the chat template. - `AutoModelForCausalLM.from_pretrained(name)`: picks the right class from `config.json` (SmolLM2 is a Llama-architecture model, so you get `LlamaForCausalLM`). - `model.generate(...)`: the decoding loop (greedy, sampling, beam search), with the KV cache handled for you. `from_pretrained` downloads into the cache (`HF_HOME`) once; later loads are local. Setting `HF_HUB_OFFLINE=1` guarantees no network access, which makes runs reproducible.

What is inside SmolLM2-135M. It is the Week 14 GPT grown up: pre-norm blocks (RMSNorm), RoPE positions (Day 93), SwiGLU MLPs, tied embeddings, and **grouped-query attention** (GQA): 9 query heads share 3 key/value heads, which shrinks the KV cache 3x (Day 111).

Forward pass and loss. `model(input_ids=x, labels=x)` returns `.loss`, the mean next-token cross-entropy. The library shifts labels internally, so you pass the *same* tensor as input and labels. Positions with label -100 are ignored (Day 103 uses this to mask prompts).

`datasets`. The `datasets` library loads Hub datasets as memory-mapped Arrow tables with `.map`, `.filter` and streaming. This week our data is small and local, so we tokenise plain tensors; the concepts transfer.

Block 2: Worked example (45 min)

Run `code/week-15/day99_hf_basics.py` (**9 s** with a warm cache). **Verified:** - **134,515,008** parameters; 30 layers, d_model 576, 9 heads, 3 KV heads, vocabulary 49,152, max positions 8,192, `rope_theta` 100,000; tied embeddings **True**; BOS and EOS are both `<|endoftext|>`. - "The quick brown fox jumps over the lazy dog." -> **10** tokens: `The`, `?quick`, `?brown`, ... (`?` marks a leading space in byte-level BPE). - Next-token distribution after "The capital of France is": ' the' **0.262**, ' Paris' **0.094**, ' located' 0.073, ' called' 0.044, ' a' 0.039. The correct fact is in the top 2, but not the top 1. - Greedy decoding of 30 tokens (**0.38 s**) loops: "the capital of the country. The capital of France is the capital of the ...". Sampling (temperature 0.8, top-k 50, seed 0) gives "the city of Paris, and the name derives from the Latin word for city ...", fluent but with an invented etymology. A 135M model is fluent before it is reliable. - Shakespeare validation perplexity (16 windows of 256): **24.42** (3.196 nats/token). Our Week 14 BPE model scored 3.470 nats/token, but on a different tokenizer, so compare in bits per character, not per token (practice problem 3).

Block 3: Coding lab (2h)

1. (30 min) Print the config and the module tree (`print(model)`). Find the GQA projection shapes: q_proj 576->576, k_proj and v_proj 576->192.
2. (45 min) Write your own greedy decoding loop with `model(...).logits` and `past_key_values`, and check it matches `generate(do_sample=False)` token for token.
3. (45 min) Compute bits per character of SmolLM2 on Shakespeare validation and compare with Week 14's 2.192.

Block 4: Practice problems (1h)

1. Why do k_proj/v_proj have 192 outputs? *Answer: 3 KV heads x 64 dims per head (576/9 = 64).*
2. Why pass `labels=input_ids` without shifting? *Answer: the model shifts internally.*
3. Convert 3.196 nats/token to bpc given ~3.10 characters per SmolLM token (Day 96). *Answer: 3.196 / 0.693 / 3.10 ~= 1.49 bpc, far better than our 2.19.*
4. Why did greedy decoding loop? *Answer: once a phrase is likely, repeating it stays likely; greedy has no randomness to escape.*

Block 5: Review (1h)

Self-check: - [ ] I can load a model and tokenizer, generate, and compute a loss. - [ ] I can read a config and recognise RoPE, GQA, tied embeddings. - [ ] I know why offline mode and model cards matter.

Extra notes: precision and memory

At fp32, 134.5M parameters take 538 MB; bf16 halves that. Inference needs the weights plus the KV cache; training needs far more (Day 100). On CPU, fp32 is the safe default; bf16 on CPU works in recent PyTorch but is not always faster.

Going deeper (optional)

Read the SmolLM2 model card on the Hub (huggingface.co/HuggingFaceTB/SmolLM2-135M) and note its training data and stated limitations.

Interview-style question (15 min, written)

"What happens when you call `AutoModelForCausalLM.from_pretrained`?" Resolve the repo, download or read cached config and weights, instantiate the class named in the config, load the safetensors weights, tie embeddings if configured.

Common mistakes / how to debug

- Mixing a tokenizer from one model with weights from another.
- Forgetting `model.eval()` / `torch.no_grad()` for inference.
- Reading a perplexity without checking context length and window count.

What to do next

Day 100: update all 134.5M weights.

Day 100: Fine-tuning a small pretrained model

Review today: Day 99 (+1), Day 97 (+3), Day 93 (+7), Day 79 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Why fine-tune? Pretraining on trillions of tokens is far beyond our budget. A pretrained model already knows language; fine-tuning on a small domain corpus moves it towards that domain's style and vocabulary. This is transfer learning (Day 75) for language models.

Full fine-tuning updates every weight with the same next-token loss used in pretraining, at a much smaller learning rate (around 1e-5 to 1e-4 rather than ~1e-3), so the model does not destroy what it knows.

Memory. With AdamW in fp32, each parameter needs 4 bytes (weight) + 4 (gradient) + 8 (two Adam moments) = 16 bytes, plus activations. For 134.5M parameters that is about **2.0 GiB** before activations. For a 7B model it would be about 112 GB, which is why full fine-tuning of large models needs several GPUs, mixed precision, and sharded optimiser states (ZeRO / FSDP), and why Day 101's LoRA exists.

Risks. - *Catastrophic forgetting:* the model gets better on the new domain and worse elsewhere (measured on Day 105). - *Over-fitting* small datasets after a few epochs. - *Learning-rate sensitivity:* too high wrecks the pretrained features quickly.

Recipe. Small learning rate, gradient clipping, a short warmup, evaluation on a held-out split before and after, and a probe set from the *original* domain to measure forgetting.

Block 2: Worked example (45 min)

Run `code/week-15/day100_full_finetune.py` (**96 s**). Settings: 100 steps, batches of 4 x 128 tokens, AdamW lr 5e-5, clip 1.0. **Verified:** - 134,515,008 parameters; fp32 weights + gradients + Adam ~= **2.00 GiB**. - Validation perplexity (8 windows): **26.29 -> 21.38** after 100 steps (0.88 s/step), from only 51,200 training tokens. - Training loss is noisy because each batch is only 512 tokens: 3.671 at step 0, 3.005 at step 40, 3.748 at step 90, 3.073 at step 99. Use the validation number, not single-batch training losses. - Sample after fine-tuning (prompt "ROMEO:"): "I will find you again. LADY CAPULET: That's well: 'tis not I, as I have been heard say, That 'tis a common thing". Compare with the Day 95 model: real words, grammatical lines, a correct character name from the same play.

Block 3: Coding lab (2h)

1. (45 min) Run with lr 5e-4 and 5e-6. Which helps validation perplexity in 100 steps, and does the higher one damage the sample quality?
2. (45 min) Freeze all but the last 4 layers and compare validation perplexity and speed.
3. (30 min) Measure peak memory with `resource.getrusage` for full fine-tuning vs inference only.

Block 4: Practice problems (1h)

1. Adam memory for 1B parameters in fp32? *Answer: ~16 GB plus activations.*
2. Why a smaller lr than pretraining? *Answer: to make small changes that preserve pretrained knowledge.*
3. Perplexity 26.29 -> 21.38 is what change in loss? *Answer: ln 26.29 - ln 21.38 ~= 3.269 - 3.062 = 0.207 nats/token.*
4. Name two ways to cut full fine-tuning memory. *Answer: bf16/mixed precision; 8-bit optimisers; gradient checkpointing; sharding.*

Block 5: Review (1h)

Self-check: - [ ] I can estimate fine-tuning memory from the parameter count. - [ ] I evaluate on held-out data before and after. - [ ] I know what catastrophic forgetting is and how to measure it.

Extra notes: gradient checkpointing

Activations for a long sequence through 30 layers can exceed the weights' memory. Gradient checkpointing stores only some layer inputs and recomputes the rest during the backward pass, trading about a third more compute for much less memory. In transformers it is `model.gradient_checkpointing_enable()`.

How much data is enough? Here 100 steps used 51,200 tokens, about one sixth of the 305,111-token training split, and perplexity still fell by 19%. Domain style is cheap to learn because the pretrained model already has the words; the fine-tune mostly shifts probabilities toward archaic forms, verse line breaks and speaker tags. New *facts* are much harder to add by fine-tuning and are better supplied at inference time with retrieval (Day 108).

Going deeper (optional)

Read the Hugging Face `Trainer` documentation and map its arguments (learning rate, warmup, gradient accumulation, eval strategy) onto our hand-written loop.

Interview-style question (15 min, written)

"Your fine-tuned model is great on the target task but worse at everything else. What happened and what can you do?" Catastrophic forgetting; lower lr, fewer steps, mix in general data, use LoRA (smaller, removable changes), evaluate on a broad probe set.

Common mistakes / how to debug

- Leaving the model in `eval()` mode, or forgetting `train()` after an evaluation.
- Judging progress from single-batch training loss.
- Saving only the weights and not the tokenizer.

What to do next

Day 101: get most of the benefit while training 0.34% of the parameters.

Day 101: LoRA from scratch, then PEFT

Review today: Day 100 (+1), Day 98 (+3), Day 94 (+7), Day 80 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Low-rank adaptation (Hu et al., 2021, "LoRA: Low-Rank Adaptation of Large Language Models"). Freeze the pretrained weight W (d_out x d_in). Learn an update ?W = B.A with A of shape r x d_in and B of shape d_out x r, with rank r much smaller than d. The layer computes

y = W x + (? / r) . B A x.

- **Initialisation:** A random, B zero, so ?W = 0 and training starts exactly at the pretrained model.
- **Scale ? / r:** keeps the update size comparable across ranks, so you can change r without retuning the learning rate much.
- **Parameters:** r.(d_in + d_out) per adapted matrix instead of d_in.d_out.
- **Merging:** after training, W' = W + (?/r).B A gives a plain model with zero inference overhead. Or keep adapters separate and swap them per task on one shared base.

Why it works. Hu et al. report that the weight updates needed for adaptation have low "intrinsic rank": a small r captures most of the benefit for many tasks. It is not guaranteed for every task; tasks that need much new knowledge benefit from higher r or full fine-tuning.

Memory benefit. Gradients and Adam states are needed only for A and B. The frozen W can even be stored in lower precision (Day 102). The forward and backward activations are still needed, so LoRA saves optimiser memory more than activation memory.

PEFT. The `peft` library wraps a model: `get_peft_model(model, LoraConfig(r, lora_alpha, target_modules, lora_dropout))`, and offers `merge_and_unload()`, saving adapters only, and many other methods (prefix tuning, IA^3 and others).

Block 2: Worked example (45 min)

Run `code/week-15/day101_lora.py` (**119 s**). LoRA r = 8, ? = 16 on q_proj and v_proj in all 30 layers, lr 1e-3, 100 steps of 4 x 128 tokens. **Verified:** - Trainable **460,800** of 134,975,808 (**0.34%**). Check: per layer, q (576->576) adds 8.(576+576) = 9,216 and v (576->192) adds 8.(576+192) = 6,144, so 15,360 x 30 = 460,800. - Logits change at init (B = 0): **0.0e+00**. The adapted model is exactly the base model before training. - My LoRA: **0.47 s/step** (vs 0.88 for full fine-tuning), validation perplexity **26.29 -> 23.19**. - Merging into W: max logit difference **7.4e-05** (float32 rounding), and the parameter count returns to 134,515,008. - `peft` with the same config: trainable **460,800**, equal to mine (**True**); after 100 steps with the same seed, validation perplexity **23.16** (mine 23.19). `merge_and_unload()` returns a plain `LlamaForCausalLM`. - Compare Day 100: full fine-tuning reached 21.38 with the same steps. LoRA on q,v recovered about 60% of that improvement ((26.29 - 23.19)/(26.29 - 21.38) ~= 0.63) with 0.34% of the trainable parameters. Day 105 tests more ranks and targets.

Block 3: Coding lab (2h)

1. (45 min) Write `LoRALinear` yourself without looking; pass the "zero change at init" and "merge equals unmerged" tests.
2. (30 min) Add LoRA dropout on the input to the A path.
3. (45 min) Save only the adapter (a few MB), reload it into a fresh base model, and confirm identical logits.

Block 4: Practice problems (1h)

1. LoRA parameters for one 4096 x 4096 matrix at r = 16? *Answer: 16.8192 = 131,072 (vs 16.8M).*
2. Why is B initialised to zero rather than A? *Answer: either makes ?W = 0, but with both zero no gradients flow; zero B + random A gives a nonzero gradient for B.*
3. What does merging cost at inference? *Answer: nothing extra; the matrix shape is unchanged.*
4. Why is the merge diff 7.4e-05, not 0? *Answer: float32 rounding: x.(W + ?W) is not bitwise equal to xW + x?W.*
5. When is serving unmerged adapters useful? *Answer: many tasks sharing one base model; hot-swapping adapters per request.*

Block 5: Review (1h)

Self-check: - [ ] I can write the LoRA forward pass, init and merge. - [ ] I can count LoRA parameters for any target set. - [ ] My implementation matches peft.

Extra notes: where to put adapters

The original paper adapted attention matrices. Later practice (for example in QLoRA, Day 102) adapts all linear layers including the MLP, which often helps more than raising r on attention alone. Day 105 tests exactly this trade-off at CPU scale.

Going deeper (optional)

Read Hu et al. (2021), section 4 (method) and section 7 (rank analysis).

Interview-style question (15 min, written)

"Explain LoRA to a colleague and when you would not use it." Frozen W plus a trainable low-rank update; cheap, swappable, mergeable. Avoid when the task needs large new knowledge or when you can afford full fine-tuning and want the last bit of quality.

Common mistakes / how to debug

- Forgetting to freeze the base: all parameters train and memory explodes.
- Wrong scale (? instead of ?/r) when comparing with peft.
- Optimiser built over all parameters including frozen ones (harmless but wasteful); build it over `requires_grad` only.

What to do next

Day 102: configure peft, and combine LoRA with a 4-bit base.

Day 102: PEFT configurations and QLoRA concepts

Review today: Day 101 (+1), Day 99 (+3), Day 95 (+7), Day 81 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Choosing a LoRA configuration. Three knobs: rank r, scale ?, and target modules. For a Llama-style block the linear layers are q, k, v, o (attention) and gate, up, down (SwiGLU MLP). Adapting more modules adds parameters roughly linearly; raising r does too.

QLoRA (Dettmers et al., 2023, "QLoRA: Efficient Finetuning of Quantized LLMs"). Key ideas: 1. Store the frozen base in **4-bit** (their NF4 data type, designed for normally distributed weights), with **double quantisation** of the scale factors. 2. Dequantise blocks on the fly to bf16 for each matmul. 3. Train LoRA adapters (in bf16) on **all linear layers**. 4. Use **paged optimisers** to survive memory spikes. The paper reports fine-tuning a 65B model on a single 48 GB GPU while matching 16-bit fine-tuning quality on their benchmarks.

Why it works. Gradients flow *through* the quantised weights to the adapters, but the quantised weights are never updated. The adapters can partly compensate for quantisation error, because they are trained with the quantised base in the loop.

Group-wise quantisation in one paragraph (Day 110 goes deeper). Split a weight matrix into groups of g values. For each group store a scale s = max|w| / 7 and integers q = round(w / s) clipped to [-8, 7] (4 bits). Dequantise as q.s. Memory: half a byte per weight plus a 16-bit scale per group (for g = 64, 0.53 bytes per weight vs 4 for fp32).

CPU honesty. The bitsandbytes 4-bit kernels used by QLoRA in practice target GPUs. Here we **simulate** 4-bit: we replace each frozen weight with its quantise-dequantise round trip (the same numerical error, but stored in fp32, so no memory is saved on this box). This shows the quality effects, not the memory savings.

Block 2: Worked example (45 min)

Run `code/week-15/day102_peft_qlora.py` (**43 s**). **Verified:**

Trainable parameters with `peft` (of 134,515,008):

targets per layer	r = 4	r = 16
q, v	230,400 (0.17%)	921,600 (0.69%)
q, k, v, o	460,800 (0.34%)	1,843,200 (1.37%)
all 7 linear	1,221,120 (0.91%)	4,884,480 (3.63%)

Simulated 4-bit (absmax, group 64) on the 7 linear layers of every block: - **106,168,320** weights quantised (78.9% of parameters; the embedding table stays fp32). - Their memory: fp32 **405 MiB**, bf16 **202 MiB**, int4 with an fp16 scale per 64 **54 MiB**. - Validation perplexity: fp32 base **26.29**, 4-bit base **36.89**. Naive round-to-nearest 4-bit hurts a small model noticeably (larger models usually tolerate it better, and better methods such as NF4, GPTQ and AWQ lose less; Day 110). - QLoRA-style: LoRA r = 8 on all linear layers (**2,442,240** trainable) over the frozen 4-bit base, 60 steps (33 s): perplexity **25.46**. That is better than the *unquantised* base (26.29). The adapters both undid the quantisation damage and adapted to Shakespeare. We did not run the fp32 + LoRA comparison at 60 steps here, so we cannot say how much quality the 4-bit base cost after training; that is lab exercise 1.

Block 3: Coding lab (2h)

1. (40 min) Run the same LoRA (all linear, r = 8, 60 steps) on the fp32 base and compare with 25.46.
2. (40 min) Try group sizes 32, 128 and per-row (one scale per output row) and plot base perplexity against bytes per weight.
3. (40 min) Implement NF4-style levels: place the 16 levels at quantiles of a standard normal instead of evenly, and compare perplexity.

Block 4: Practice problems (1h)

1. Memory of a 7B base at 4-bit with fp16 scales per 64? *Answer: 7e9 x (0.5 + 2/64) bytes ~= 3.7 GB.*
2. Why can't we update the 4-bit weights directly? *Answer: they are discrete; tiny gradient steps round away. Adapters stay in high precision.*
3. Why does adapting all linear layers cost 2.65x the parameters of q,k,v,o at the same r? *Answer: the MLP matrices are larger (576 ? 1,536).*
4. Why might the quantised model + LoRA beat the fp32 base? *Answer: the adapters were trained on the target domain; the base was not.*

Block 5: Review (1h)

Self-check: - [ ] I can list QLoRA's four ingredients. - [ ] I can compute memory for a quantised base. - [ ] I can say exactly what my CPU simulation does and does not show.

Extra notes: other PEFT methods

Prefix/prompt tuning learns virtual tokens prepended to every layer's keys and values; IA^3 learns per-channel scaling vectors; adapters (Houlsby et al., 2019) insert small bottleneck MLPs. LoRA dominates in practice because it merges away at inference and is simple to apply.

Going deeper (optional)

Read Dettmers et al. (2023), section 3, on NF4 and double quantisation.

Interview-style question (15 min, written)

"You need to fine-tune a 13B model on one 24 GB GPU. Plan it." 4-bit base (~7 GB), LoRA on all linear layers, bf16 compute, gradient checkpointing, small micro-batch with gradient accumulation, paged optimiser.

Common mistakes / how to debug

- Target-module names that match nothing: peft errors out or you train zero parameters; always print trainable parameters.
- Quantising the embedding/head (often kept in higher precision).
- Expecting memory savings from a *simulated* quantisation.

What to do next

Day 103: teach a base model to follow instructions.

Day 103: Instruction tuning and chat templates

Review today: Day 102 (+1), Day 100 (+3), Day 96 (+7), Day 82 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Base vs instruct models. A base model continues text. Ask it a question and it may continue with more questions, as Day 99's greedy loop showed. An *instruct* (chat) model has been further trained to answer. Instruction tuning is the first step of that post-training.

Supervised fine-tuning (SFT). Collect (instruction, response) pairs and fine-tune with the next-token loss on the formatted conversation. Wei et al. (2022), "Finetuned Language Models Are Zero-Shot Learners" (FLAN), and Ouyang et al. (2022), "Training language models to follow instructions with human feedback" (InstructGPT), established the recipe: SFT on demonstrations, then preference optimisation (Day 106).

Chat templates. Conversations are serialised with special tokens. SmolLM2-Instruct uses the ChatML style: `<|im_start|>system ... <|im_end|>`, `<|im_start|>user ... <|im_end|>`, `<|im_start|>assistant`. The template lives in the tokenizer (`tokenizer.apply_chat_template`). Use the *exact* template the model was trained with; a mismatch silently degrades quality. `add_generation_prompt=True` appends the assistant header so the model starts answering.

Prompt-loss masking. Usually only the response tokens are trained: set the labels of system and user tokens to -100. Arguments for it: the model should learn to answer, not to generate user questions; and long prompts would otherwise dominate the loss. Arguments against: for short prompts it may matter little, and some work finds that training on prompts also helps. We test it.

Stop tokens. The response must end with `<|im_end|>` in training, and generation must stop on that token. Our first run forgot to pass it as an EOS token to `generate`: the model produced the right words and then rambled ("window tiger coffeeaidaidaid"), scoring 0.07. A one-line fix (`eos_token_id=[..., im_end]`) is the difference between a broken and a working evaluation.

Our task. Reversing the order of 3-5 words from a 40-word list ("Reverse the order of these words: red fish blue" -> "blue fish red"). It is synthetic, automatically checkable, and something a 135M base model cannot do zero-shot.

Block 2: Worked example (45 min)

Run `code/week-15/day103_instruction_tuning.py` (about 6 minutes). 2,000 training examples; 40 held-out prompts not in the training set; LoRA r = 16 on all linear layers, lr 1e-3, 250 steps of 8 examples. **Verified:** - The rendered template contains a default system prompt ("You are a helpful AI assistant named SmolLM, trained by Hugging Face"), then the user turn, then `<|im_start|>assistant`. - One example is **48** tokens, of which only **6** (the answer plus `<|im_end|>`) count in the masked loss. - Zero-shot exact match on the 40 held-out prompts: base **0.00**, SmolLM2-135M-Instruct **0.00**. - After SFT with prompt masking: exact match **1.00** (135 s). Sample: "coffee tiger window" -> **"window tiger coffee"**. - After SFT *without* masking: also **1.00** (132 s).

So on this task the masking choice made no difference: the prompt is highly regular, and 250 steps were enough either way. Report it as a null result for this setting, not as evidence that masking never matters. Harder tasks with varied, long prompts are where masking is usually argued to matter.

Block 3: Coding lab (2h)

1. (45 min) Cut training to 50 and 100 steps and compare masked vs unmasked again: does a gap appear early?
2. (45 min) Make the test harder: 6-8 words, including words not in the training list. What breaks?
3. (30 min) Render a multi-turn conversation with `apply_chat_template` and build labels that train *only the last* assistant turn.

Block 4: Practice problems (1h)

1. What fraction of tokens is trained with masking in our example? *Answer: 6/48 = 12.5%.*
2. Why did the instruct model score 0 zero-shot? *Answer: a 135M model cannot reliably follow this instruction; it was never trained on it.*
3. What is the effect of forgetting the stop token in evaluation? *Answer: correct answers followed by junk fail exact match.*
4. Why use a synthetic task? *Answer: unlimited data, exact automatic grading, no licence issues.*

Block 5: Review (1h)

Self-check: - [ ] I can render a chat template and explain each special token. - [ ] I can build masked labels. - [ ] I check stop tokens before trusting an evaluation.

Extra notes: real SFT data

Public instruction datasets range from human-written (such as Databricks' Dolly) to model-generated (such as Alpaca-style data), and modern recipes mix many sources with filtering. Quality and diversity matter more than size: Zhou et al. (2023), "LIMA: Less Is More for Alignment", report strong results from about 1,000 carefully curated examples. Always check the licence, because some datasets generated with commercial models forbid certain uses.

Going deeper (optional)

Read the chat-templating page of the Hugging Face transformers documentation.

Interview-style question (15 min, written)

"Your fine-tuned chat model keeps generating the user's next turn after its answer. Why?" Missing `<|im_end|>` in training targets, or the stop token not set at inference, or a template mismatch.

Common mistakes / how to debug

- Training on one template and serving another.
- Masking the end-of-turn token, so the model never learns to stop.
- Test prompts that also occur in the training set (we filter them out).

What to do next

Day 104 evaluates this model more broadly, including what it lost.

Day 104: Evaluating LLMs and the fine-tuned model

Review today: Day 103 (+1), Day 101 (+3), Day 97 (+7), Day 83 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Evaluate on several axes. A fine-tune is a trade. Measure: 1. **The target task**, with an exact metric where possible (exact match, unit tests, F1). 2. **Retention:** perplexity or tasks from the original domain, to detect forgetting. 3. **General ability:** standard benchmarks.

Perplexity is cheap and sensitive but only comparable with the same tokenizer and text, and it does not measure helpfulness.

Multiple-choice benchmarks (MMLU, HellaSwag, ARC and others) are often scored by **log-likelihood**: compute log P(option | question) for each option and pick the highest. Choices matter: summing token log-probs penalises long options; length normalisation (mean per token) or per-character normalisation counteracts that. Different harnesses make different choices, so the same model can get different scores. Always report how it was scored.

Generative evaluation checks free-form answers by exact match, by execution (code), or by a judge.

LLM-as-a-judge (Zheng et al., 2023, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena") uses a strong model to grade or compare answers. It scales, but it has known biases: **position bias** (preferring the first or second answer), **verbosity bias**, and **self-preference**. Mitigation: evaluate both orders and count only consistent verdicts, use rubrics, and check agreement with humans on a sample.

Contamination. Benchmark questions leak into web-scale pretraining data, which inflates scores. Prefer held-out or freshly written tests for decisions that matter.

Block 2: Worked example (45 min)

Run `code/week-15/day104_eval_finetuned.py` (**35 s**; needs the Day 103 adapter). Models: base, SmolLM2-135M-Instruct, and base + the Day 103 SFT adapter (loaded with 0 unexpected keys). **Verified:**

	base	instruct	base + SFT LoRA
reverse-words exact match (40)	0.00	0.00	**1.00**
Shakespeare val perplexity	26.29	32.05	**57.90**
16-question MC, sum log-prob	0.88	0.88	0.88
16-question MC, length-normalised	0.81	0.81	0.81

- **Forgetting is real:** the SFT adapter more than doubled Shakespeare perplexity (26.29 -> 57.90) while learning the task perfectly. The instruct model is also worse than the base at plain text continuation (32.05), a common side-effect of chat post-training.
- **The small MC set did not separate the models.** All three got the same scores. With 16 questions, one question is 6 percentage points, so this set is too small to detect anything but huge differences. Length normalisation *lowered* accuracy here (0.81 vs 0.88), so it is not automatically better. The questions were written for this course (basic facts, 4 options), not taken from a benchmark.
- **Position bias, demonstrated.** Using the instruct model as a judge ("Which statement is true? A ... B ..."), on 16 pairs in both orders: it chose A in **10 of 32** cases, gave the *same* verdict after swapping in **0 of 16** pairs (always the same letter regardless of content), and was correct **16 of 32** times, exactly chance. A 135M model is far too weak to judge, and this is the failure mode the swap test exists to catch.

Block 3: Coding lab (2h)

1. (45 min) Write 50 more MC questions and report accuracy with a 95% binomial confidence interval. Are the models now separable?
2. (45 min) Re-run the judge with the order instruction "Answer: B" style suffix changed; does the bias flip?
3. (30 min) Add per-character normalisation to the MC scorer and compare.

Block 4: Practice problems (1h)

1. Standard error of accuracy 0.88 on 16 questions? *Answer: sqrt(0.88.0.12/16) ~= 0.081.*
2. What does a judge consistency of 0/16 mean? *Answer: its verdicts depend only on position.*
3. Why might length normalisation hurt? *Answer: it can favour long options whose later tokens are easy to predict.*
4. Name two ways to reduce the forgetting seen here. *Answer: mix general text into SFT data; lower lr / fewer steps / smaller rank; keep adapters separate and switch them off when not needed.*

Block 5: Review (1h)

Self-check: - [ ] I measure task, retention and general ability. - [ ] I can score multiple choice by log-likelihood and state the normalisation. - [ ] I run the swap test before trusting a judge.

Extra notes: evaluation harnesses

EleutherAI's lm-evaluation-harness and Hugging Face's lighteval implement many benchmarks with fixed prompts and scoring rules, which makes numbers comparable *within* a harness. When a model card reports a benchmark, look for the harness, the number of shots, and the normalisation.

Going deeper (optional)

Read Zheng et al. (2023), section 3 on judge biases.

Interview-style question (15 min, written)

"How would you evaluate a fine-tuned support chatbot?" Task metric on held-out real tickets, retention checks, safety tests, judge with swap test and human spot checks, and online A/B metrics.

Common mistakes / how to debug

- Reporting one tiny test set without error bars.
- Using an LLM judge without checking position bias.
- Comparing perplexities across tokenizers.

What to do next

Day 105: compare full fine-tuning and LoRA variants on one budget.

Day 105: PROJECT W15: full fine-tune vs LoRA; Quiz 15

Review today: Day 104 (+1), Day 102 (+3), Day 98 (+7), Day 84 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Project question. With the same data and number of steps, how do full fine-tuning and LoRA variants compare in target quality, trainable parameters, time and forgetting?

Design. One script (`day105_project_lora.py`), one seed, Shakespeare as the target domain, and **this course's own notes** (`weeks/week-01.md`, 12,705 SmolLM tokens) as the forgetting probe: modern technical English, very different from Shakespeare. Same 80 steps of 4 x 128 tokens for every run. Full fine-tuning at lr 5e-5 (Day 100); LoRA at lr 1e-3 (Day 101).

Confounders to keep in mind. The learning rates differ (each is a typical value for its method), and there is one seed. So the comparison is "typical settings", not "equal settings". A fair study would tune the learning rate per method and average seeds.

Deliverables. The results table, a 300-word report with a recommendation, and Quiz 15.

Block 2: Worked example (45 min)

Run `code/week-15/day105_project_lora.py` (**250 s** total). **Verified:**

run	trainable	Shakespeare val ppl	notes ppl	time (80 steps)
no training	0	26.29	24.89	-
full fine-tune, lr 5e-5	134,515,008	**21.34**	26.29	59 s
LoRA q,v r = 2	115,200	23.23	27.11	48 s
LoRA q,v r = 8	460,800	23.02	28.29	35 s
LoRA q,v r = 32	1,843,200	22.68	30.17	38 s
LoRA all-linear r = 8	2,442,240	21.42	28.65	51 s

Findings: - **Target quality:** full fine-tuning was best (21.34), with LoRA on all linear layers nearly equal (21.42) using 1.8% of the trainable parameters. On q,v only, higher rank helped modestly (23.23 -> 22.68 from r = 2 to 32). *Where* you adapt mattered more than rank: all-linear r = 8 beat q,v r = 32 with a similar parameter count. - **Forgetting:** every run got worse on the course notes. Here full fine-tuning forgot *least* (24.89 -> 26.29), and LoRA forgetting grew with rank (27.11 -> 30.17). That is the opposite of the common intuition that LoRA forgets less; Biderman et al. (2024), "LoRA Learns Less and Forgets Less", report that pattern at much larger scale. The likely cause here is the learning rate: LoRA ran at 1e-3, 20x higher than full fine-tuning. This is why the confounder note above matters. - **Time:** LoRA steps were faster (35-51 s vs 59 s) but not dramatically, because the forward and backward passes through the frozen model still dominate on CPU. Timings varied between runs (r = 2 took longer than r = 8), so do not over-read them.

Block 3: Coding lab (2h)

1. (60 min) Remove the confounder: run LoRA all-linear r = 8 at lr 2e-4 and 5e-4, and full fine-tuning at 2e-5 and 2e-4. Plot val ppl against notes ppl for all runs (a trade-off curve).
2. (30 min) Add a second seed to the best two runs.
3. (30 min) Write the 300-word report: recommendation, evidence, caveats.

Block 4: Practice problems (1h)

1. Which run gives the best target ppl per trainable parameter? *Answer: LoRA q,v r = 2 (23.23 with 115,200).*
2. Why didn't LoRA give a big speed-up? *Answer: compute is dominated by passes through the frozen 134.5M-parameter network; LoRA mainly saves optimiser memory.*
3. What single change would most improve the fairness of the comparison? *Answer: tuning the learning rate per method.*
4. When would you still choose LoRA even if full fine-tuning is slightly better? *Answer: limited memory, many tasks on one base, small adapter files.*

Block 5: Review (1h)

Self-check: - [ ] I can design a comparison with a forgetting probe. - [ ] I can name the confounders in my own experiment. - [ ] I took Quiz 15 and checked my answers.

Extra notes

This week's whole arc ran on a CPU with a 135M-parameter model in minutes per experiment. The same scripts work unchanged on a GPU with a larger model (for example a 1-2B parameter model) by moving tensors to `cuda` and using bf16, at which point real 4-bit QLoRA via bitsandbytes becomes available and runs of thousands of steps are practical.

Going deeper (optional)

Read Biderman et al. (2024) and compare their learning-rate choices with ours.

Interview-style question (15 min, written)

"Full fine-tuning or LoRA for a new product?" Start with LoRA on all linear layers (cheap, swappable), measure target and retention, and escalate to full fine-tuning only if the target gap matters and you can afford it.

Common mistakes / how to debug

- Comparing methods at different learning rates and attributing the difference to the method.
- No retention probe.
- One seed reported as a conclusion.

What to do next

Week 16: DPO, embeddings, retrieval and RAG.

Weekly quiz: Week 15

1. Why does SmolLM2's k_proj have 192 output features?
2. What does `labels=-100` do?
3. Estimate fp32 AdamW memory for full fine-tuning of 134.5M parameters.
4. Write the LoRA forward pass and its initialisation.
5. How many trainable parameters does LoRA r = 8 on q and v add to SmolLM2-135M?
6. Why is the merged LoRA output not bitwise identical?
7. Name QLoRA's main ingredients.
8. What did simulated 4-bit do to base perplexity, and what did LoRA on top achieve?
9. Why did our first instruction-tuning evaluation score only 0.07?
10. Did prompt-loss masking matter on our task?
11. What did the judge swap test show?
12. In the project, which run forgot least, and what confounder may explain it?

Answers

1. GQA: 3 KV heads x 64 dimensions.
2. Excludes that position from the loss.
3. 16 bytes per parameter ~= 2.0 GiB, plus activations.
4. y = Wx + (?/r).B A x, with A random and B zero so the model starts unchanged.
5. 460,800 (15,360 per layer x 30).
6. Floating-point rounding (max logit difference 7.4e-05).
7. 4-bit NF4 base, double quantisation, LoRA on all linear layers, paged optimisers.
8. 26.29 -> 36.89; LoRA r = 8 all-linear for 60 steps brought it to 25.46.
9. Generation did not stop at `<|im_end|>`, so correct answers were followed by junk.
10. No: both masked and unmasked reached 1.00 exact match.
11. 0 of 16 verdicts survived swapping; accuracy 16/32, exactly chance: pure position bias.
12. Full fine-tuning (notes ppl 24.89 -> 26.29); LoRA used a 20x higher learning rate.